COSE361 Final Project Report

2023320340 Seoyeon Cho

1. Introduction

The primary characteristic of this code is its dynamic switching between offensive and defensive roles, unlike the baseline which separates attackers and defenders. If both agents on the opposing team are attacking, both agents on our team will defend. Conversely, if both agents on the opposing team are defending, our agents will switch to offense. If one opposing agent is defending, our agent closest to that defender will take on the defensive role, while the other agent will attack. In cases where both opposing Pac-Men invade, in 'your_baseline2', one of our Pac-Men guards the capsule area while the other chases the nearest enemy, effectively protecting our territory. Additionally, if one of our agents consumes a capsule, the other agent will switch to Pac-Man mode to focus on consuming food. This role transition is well-implemented in both 'your_baseline1' and 2.

2. Methods

2.1. The commonality between 'your_baseline1' and 'your_baseline2'

When trying to consume a large amount of food, entering the maze too deeply resulted in losing points. To prevent this, although an ideal algorithm could evade ghosts perfectly and score highly, it was practically challenging to implement. Instead, the code was set to consume only 4-5 food items at a time, allowing Pac-Man to quickly increase the score by targeting easily accessible food first. Various functions were used to set target points, stored in a global dictionary target as 'target[self.index]=(x,y).' The breadth first search function was then used to find the optimal path to these target points. Setting target as a global variable helped efficiently track the goals of our team's other agents.

When determining the direction of movement, functions targeting the nearest enemy Pac-Man, the nearest boundary of our/opposing territory, and the nearest food were executed. The path to these target points was found using the 'bfs function'. Goals were dynamically adjusted according to changing situations, and if our two agents had overlapping targets, the agent farther from the goal would change its target. To change the target, the 'gameState' was temporarily altered to recognize the goal as a wall, and a new target was then found.

2.2. Differences between 'your_baseline1' and 'your_baseline2'

The primary difference between 'your_baseline1' and 'your_baseline2' lies in the logic for escaping from opposing ghosts. 'your_baseline2', identified as my best code, introduces a flee mode.

In 'your_baseline1', movement towards a closer enemy was restricted when the enemy was within a certain distance. This escape logic was ineffective, causing Pac-Man to be easily caught, resulting in a more aggressive behavior.

In 'your_baseline2', a flee mode exists. When the nearest enemy is within a certain distance, our agent targets the closest point in our territory and sets 'self.alert' to 1. The 'self.alert' resets to 0 upon reaching the goal. While 'self.alert' is 1, the target does not change, allowing effective retreat towards our territory without deviation. This method proved to be the most efficient for escaping. Multiple trials indicated that quickly consuming a large amount of food after consuming a capsule was crucial for winning. Therefore, if a capsule was within a certain distance, one of the two Pac-Men would always target it. The threshold distance for triggering flee mode was optimized through several trials.

In addition, 'your_baseline2' includes several minor improvements.

2.2. Features of your_baseline3

'your_baseline3' sets both agents as defenders based on the default baseline code. It aims to result in a tie by focusing solely on defense and abandoning scoring through offense. When 'your_baseline2' operated in the opposing territory, early versions struggled to avoid opposing ghosts, leading to the creation of your_baseline3 to check the possibility of winning when the opponent focused solely on defense.

However, ‘your_baseline2’'s high threshold for initiating flee mode meant Pac-Men did not easily invade the opposing territory, often resulting in ties, although the final version of ‘your_baseline2’ often won.

3. Results

	your_best(red)
	<Average Winning Rate>
your_base	0.5
your_base	0.6
your_base	0.7
baseline	0.8
Num_Win	4
Avg_Winn	0.65
	<Average Scores>
your_base	1.9
your_base	2.6
your_base	2.5
baesline	3.1
Avg_Score	2.525

4. Conclusion (Free Discussion)

4.1. Limitations of ‘your_best’

When the opposing Pac-Men consume a capsule, our team's Pac-Men's response is unnatural. They need to prevent the opposing Pac-Men from escaping our territory but fail to do so effectively, requiring significant improvement. Additionally, referencing other codes, some Pac-Men hovered around the nearest food in the opponent's territory. This caused our Pac-Men to pursue the opposing Pac-Men directly, leading to missed opportunities to change goals efficiently. Furthermore, when both teams' Pac-Men consume capsules simultaneously, there is confusion over whether to switch to defense or offense, resulting in missed opportunities to consume food.

Given the uncertainty of whether the opponent's algorithm focuses on offense or defense, dynamically switching between roles was intended, but a fixed algorithm with one attacker and one defender often proved more efficient. The code does not utilize a weighting function but relies solely on environmental changes to move Pac-Man. Using a weighting function could have potentially resulted in a more efficient code, leaving a sense of regret.